UBA

Reconocimiento de Patrones en Imágenes Tipográficas: Análisis Exploratorio y Clasificación Supervisada

Laboratorio de Datos
Marzo 2026

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

Integrantes:
Tobías Passarelli
Santiago Pizzani Esteban

Docentes:
Pablo Turjanski
Manuela Cerdeiro
María Clara Fontana

Resumen

En este trabajo se aborda el problema de la clasificación de caracteres del alfabeto inglés a partir de imágenes tipográficas. Se realizó un análisis exploratorio de datos (EDA) sobre un dataset de 26.416 instancias, confirmando un balance perfecto entre las 26 clases. Se implementaron dos aproximaciones de modelado supervisado: una clasificación binaria (letras ”**O**”vs. ”**L**”) mediante el algoritmo K-Nearest Neighbors (KNN), y una clasificación multiclase para la totalidad del alfabeto utilizando Árboles de Decisión. Mediante validación cruzada y una partición de datos en tres etapas (Entrenamiento, Validación y Held-out), se optimizaron los hiperparámetros de los modelos. El modelo final basado en árboles de decisión alcanzó una exactitud del 77.89 % en datos no vistos, demostrando la viabilidad del enfoque propuesto para el reconocimiento de caracteres.

1. Introducción

El presente trabajo busca caracterizar el conjunto de datos llamado *English Typed Alphabet*, previo a la implementación de modelos de aprendizaje automático. El dataset está compuesto por imágenes de caracteres tipográficos en escala de grises, donde cada fila representa un píxel de la imagen transformada en vector, en el mismo se incluyen las 26 letras del alfabeto inglés, capitalizadas, y con múltiples versiones de cada letra.

Para lograr dicho objetivo, se buscará crear dos modelos de aprendizaje supervisado aplicando el modelo de K-Nearest Neighbors (KNN) y el de Árboles de Decisión, por lo que vale la pena ahondar en el marco teórico existente detrás de cada uno de ellos. [2]

1.1. Modelo K-Nearest Neighbors (KNN)

Se trata de uno de los algoritmos de aprendizaje supervisado más intuitivos del machine learning. Su funcionamiento es relativamente sencillo, y consiste en que un dato nuevo se clasifica según los datos más cercanos a él, esto se representa mediante un espacio de n-dimensiones diseñado a partir de los datos de entrenamiento [1]. El modelo memoriza la posición de cada uno de los puntos de ese set de entrenamiento, y al ingresar un nuevo dato que requiere ser clasificado, se seleccionan los K vecinos más cercanos, esa cercanía se calcula mediante la distancia euclidiana, fórmula disponible en la Ecuación 1, donde a_r es el valor del atributo de un punto dado, con x_i y X_j puntos en el conjunto de datos.

$$d(x_i, x_j) \equiv \sqrt{\sum_{r=1}^{n} (a_r(x_i) - a_r(x_j))^2} \tag{1}$$

La clase asignada al nuevo dato corresponde al valor más frecuente entre los K vecinos seleccionados, es decir, se determina por votación mayoritaria [1]. El parámetro K juega un rol fundamental en el comportamiento del modelo: cuando $K = 1$, el algoritmo simplemente asigna al nuevo dato la clase del vecino más cercano. Para valores mayores de K, se considera el valor más común entre los K ejemplos de entrenamiento más próximos [1].

1.2. Modelo de Árbol de Decisión

Esta clase de modelos se caracterizan por realizar una serie de preguntas al conjunto de datos para así poder clasificar nueva información. Se construyen mediante una sucesión de preguntas si/entonces, lo que lleva al modelo a tomar una decisión [2].

Estructuralmente, un árbol de decisión se compone de nodos, ramas y hojas. Cada nodo determina una prueba sobre algún atributo del dato a clasificar, cada rama representa uno de los posibles valores de ese atributo, y las hojas corresponden a los nodos terminales que determinan la clase asignada [1]. Además, la profundidad del árbol indica la máxima cantidad de preguntas realizadas para llegar a una decisión.

El proceso de clasificación comienza en el nodo raíz, donde se evalúa el atributo correspondiente y se desciende por la rama asociada al valor del dato. Este proceso se repite en cada nodo hasta alcanzar una hoja [1].

1.3. Exactitud y Complejidad de un Modelo

Los modelos entrenados usan dos tipos de datos: Datos de entrenamiento y datos de prueba (test). Para evaluar qué tan bien se desempeña el modelo al clasificar los datos se usa la métrica de la exactitud, esta nos dice qué tan capaz es el modelo de asignarle la clasificación correcta a un dato. Existe un balance entre qué tan exacto es el modelo sobre los datos de entrenamiento respecto a la exactitud que tiene al momento de generalizar para nuevos datos [2]. Un modelo demasiado exacto en los datos de entrenamiento suele presentar sobreajuste (overfitting), mientras que uno con baja exactitud se dice que presenta subajuste (underfitting). Un ejemplo de esto se puede apreciar en la Figura 1, donde se observa un punto ideal donde la exactitud para los datos generalizados encuentra un balance con la complejidad del modelo.

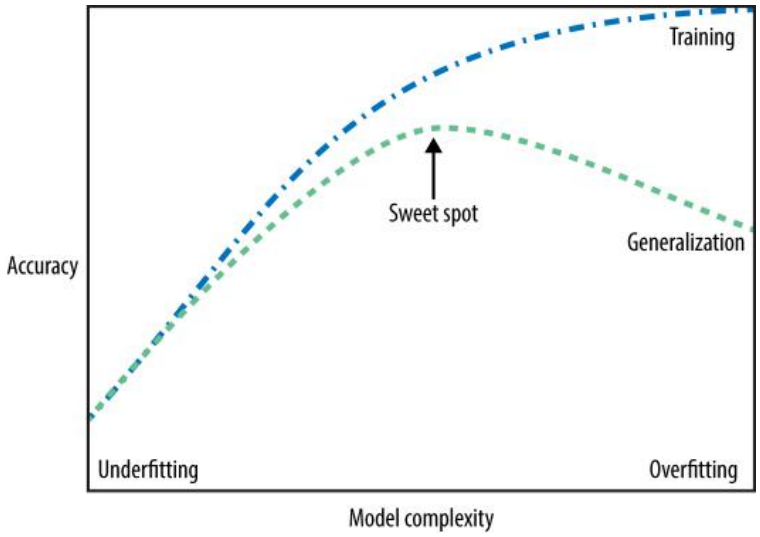

Figura 1: Esquema del balance entre complejidad del modelo y exactitud en los resultados. La curva azul presenta la exactitud para datos con los que se entrenó el modelo, mientras que la curva verde es la exactitud al generalizar los resultados del entrenamiento a un conjunto externo de datos (nuevos datos). Figura extraída de [2].

Adicionalmente, se utilizó la varianza como medida estadística de dispersión, la cual cuantifica cuánto se alejan los valores de un conjunto de datos respecto a su media. Esta se define en la

Ecuación 2, donde σ^2 es la varianza, x_i un dato, $\bar{x}$ la media del conjunto y n la cantidad de datos.

$$\sigma^2 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n} \tag{2}$$

1.4. Metodología de Trabajo

El estudio se estructuró en tres fases incrementales de complejidad:

- **Análisis Exploratorio y Distancias:** Evaluación de la variabilidad interna de los caracteres y cálculo de distancias euclídeas para medir la similitud entre morfologías (ej. O vs. Q).
- **Clasificación Binaria (KNN):** Implementación de un modelo de vecinos más cercanos para distinguir entre O y L, analizando el impacto de la selección de atributos según su varianza.
- **Clasificación Multiclase (Árboles de Decisión):** Entrenamiento de modelos para la totalidad del alfabeto, utilizando técnicas de Validación Cruzada (K-Folding) para la selección de hiperparámetros y un conjunto de evaluación final (Held-out) para medir la capacidad de generalización.

2. Análisis Exploratorio de Datos

Antes del modelado, se analizó la estructura espacial y la variabilidad del dataset para comprender la naturaleza del problema de clasificación. Para ello, se empezó por llevar a cabo una auditoría inicial de los datos disponibles, lo que arrojó los siguientes resultados:

- **Volumen de datos:** 26.416 instancias.
- **Dimensiones:** Cada imagen representa una letra en mayúscula del abecedario inglés; cuenta con 784 atributos representados en una imagen rectangular de 28×28 píxeles en escala de grises. Es decir, las celdas de los píxeles pueden tomar valores enteros entre 0 y 255.
- **Balance de clases:** El conjunto está perfectamente balanceado, con 1.016 muestras por cada una de las 26 letras del alfabeto. Esto permite que la métrica de Exactitud (Accuracy) sea un indicador fiel del rendimiento del modelo sin sesgos por frecuencia de clase.

Que el dataset se encuentre compuesto por imágenes ciertamente representa un reto importante para su análisis. A diferencia de otras tablas donde cada columna tiene un significado claro por sí sola (como una edad o un precio), aquí la información está repartida en cientos de píxeles. Un píxel aislado no dice nada; solo se puede entender que hay una letra cuando se observa la relación que tienen todos los píxeles entre sí para formar una figura. Además, al estar en escala de grises, cada píxel se representa con un solo número, pero si las imágenes fueran a color la cantidad de datos se triplicaría por la necesidad de considerar el esquema de colores RGB, volviendo el análisis aún más tedioso. Por último, la exploración es difícil porque no se puede simplemente leer una tabla de números para encontrar errores, uno se ve obligado a generar gráficos de las imágenes para saber qué contienen. Revisar manualmente miles de estas figuras para detectar fallos requeriría una inversión de tiempo y memoria que muchas veces no es viable, sobretodo si no se cuenta con un sistema de filtros como el que puede existir en otro tipo de datasets. Es por todos esos motivos que a continuación se usarán múltiples métricas para poder entender el conjunto de datos con el que se entrenarán los modelos, y poder planear acordemente considerando aquellas peculiaridades encontradas en las letras del dataset.

2.1. Varianza de Píxeles y Exploración de Atributos

Para ver qué parámetros influyen a la hora de predecir a qué letra corresponde cada imagen, se decidió calcular la varianza de cada uno de los 784 píxeles en todas las instancias del dataset. Este análisis se puede observar en la Figura 2, donde se muestra que la información clave para clasificar las letras se encuentra en la parte central del cuadrante de 28×28 píxeles. Se observó que los bordes presentan varianza nula o cercana a cero, indicador de que los píxeles cercanos a los bordes no aportan información útil para la clasificación. Por otra parte, el centro de la figura presenta mayores niveles de varianza, diferenciando los trazos característicos de cada letra.

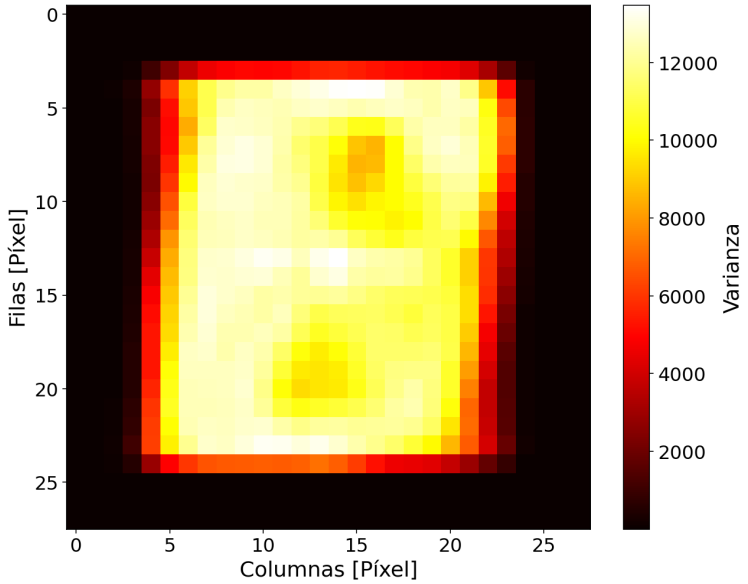

Figura 2: Mapa de calor con la varianza por píxel para la totalidad del dataset. Los píxeles de los bordes presentan varianza cercana a cero, mientras que la región central posee la mayor varianza.

Tras identificar que la información relevante para distinguir letras se concentra en la región central de la imagen, se procedió a analizar la variabilidad interna de una misma letra. Para ello, se tomó la letra J como caso de estudio. Se empezó por calcular la imagen promedio de la letra, obtenida al promediar el valor de cada píxel a lo largo de todas las instancias de J presentes en el dataset, resultando así en una representación "ideal" del carácter. A partir de esta imagen de referencia, se calculó la distancia euclidiana de cada instancia respecto a dicho promedio.

Los resultados se muestran en la Figura 3: **(3a)** presenta una muestra representativa de las variaciones de la letra J en el dataset; **(3b)** expone el mapa de calor de la varianza por píxel considerando todas las instancias de J; mientras que en **(3c)** se exhibe el histograma de distancias euclidianas respecto a la imagen promedio, calculadas mediante la Ecuación 1. La distribución resultante es aproximadamente unimodal con media 1677.5, sugiriendo que la mayoría de las instancias presentan una similitud consistente respecto al prototipo ideal, siendo este la J promedio.

Finalmente, se extendió este análisis comparativo entre distintas letras del dataset. Para cada par, se generó un carácter promedio representativo de cada clase y se calculó la distancia euclidiana entre ellos. Los resultados, presentados en la Figura 4, se ordenan de menor a mayor distancia, donde un valor menor refleja semejanza visual. Destaca el par D y O como el de mayor similitud gracias a su forma circular compartida, mientras que S y M resultaron tener la mayor separación en distancia por ser las letras más fáciles de diferenciar entre los casos elegidos.

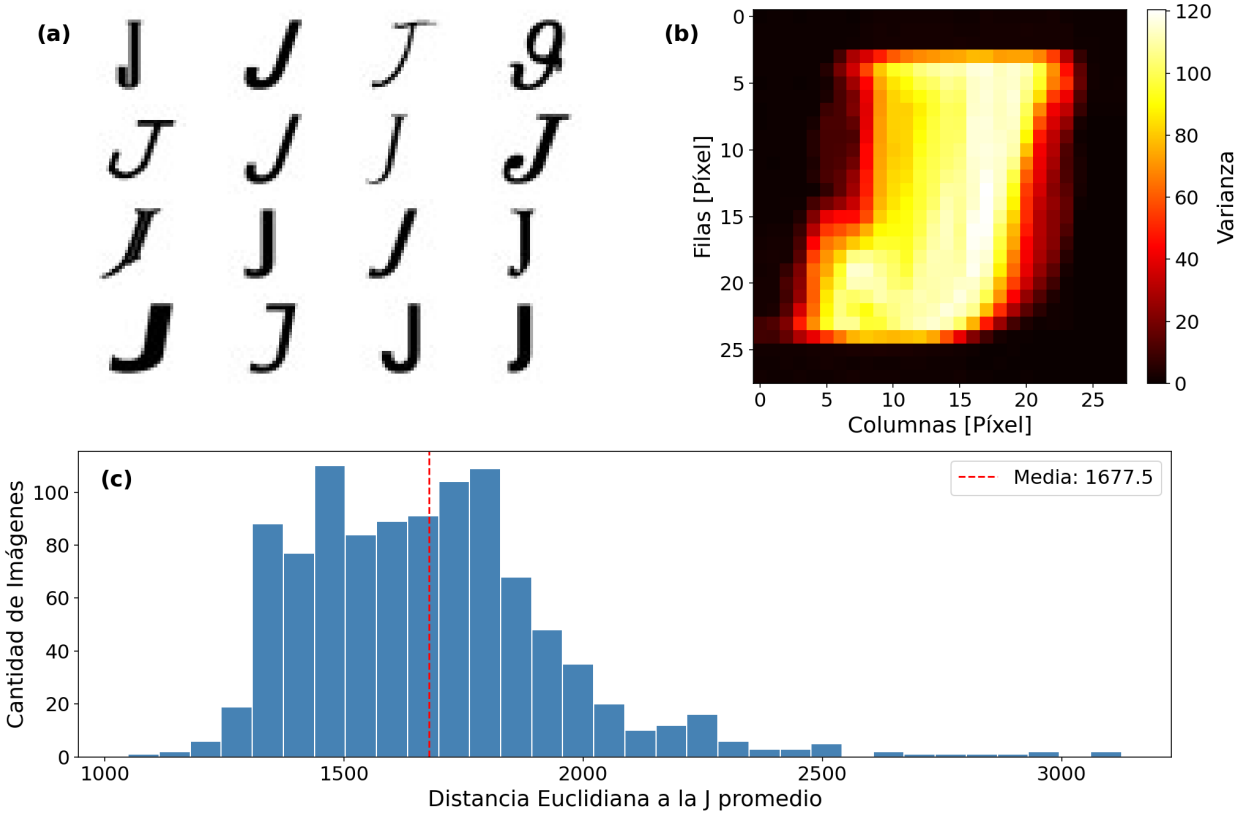

Figura 3: Variación existente entre las imágenes para la letra J. **(a)** Distintas presentaciones para la letra J presentes en el dataset. **(b)** Mapa de calor con la varianza por píxel. **(c)** Distancia Euclidiana a la J promedio, calculada como el píxel promedio entre todas las imágenes del carácter.

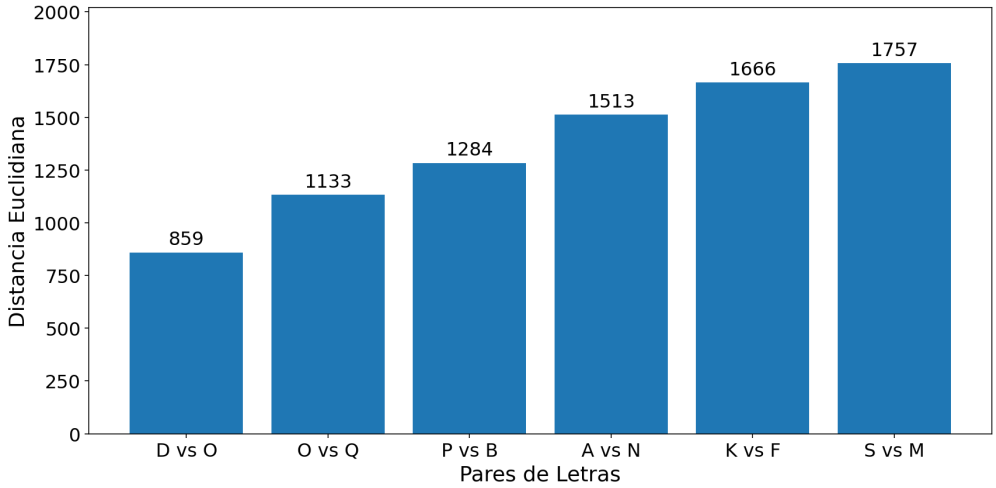

Figura 4: Distancia euclidiana para distintos pares de letras: a mayor distancia, más se diferencian entre sí. Se observa que D y O son las más parecidas, mientras que S y M son las más diferentes.

3. Clasificación Binaria

En esta sección se abordó el problema de separar dos clases con diferencias morfológicas evidentes: la O y la L, utilizando el algoritmo K-Nearest Neighbors (KNN). Se tomaron todas las instancias correspondientes a ambas letras conformando un total de 2032 elementos, este conjunto se dividió en datos de entrenamiento y de prueba, con una proporción del $75\,\%$ y $25\,\%$ respectivamente. Se reordenaron los datos de forma aleatoria, y se aplicó una estratificación para mantener la misma proporción de letras en los dos sets. Para evitar el sesgo de datos, las transformaciones y el cálculo de varianzas se realizaron exclusivamente sobre el conjunto de entrenamiento.

3.1. Impacto de la Selección de Atributos

Para evaluar la importancia de la información espacial, inicialmente se entrenó un modelo KNN con $K = 3$ incrementando progresivamente la cantidad de píxeles utilizados para el mismo. Específicamente, se conformaron subconjuntos de 3, 10, 50, 100 y 200 píxeles. El desempeño se midió a través de la métrica de exactitud, que representa el porcentaje de clasificaciones correctas para los caracteres. Los resultados se observan en la Figura 5, demuestran que la exactitud no no depende únicamente de la cantidad de píxeles utilizados para entrenar el modelo, sino también cuáles son los elegidos para realizar esa tarea. Para probar eso, se contrastó el rendimiento al entrenar con los píxeles de mayor varianza frente a los de menor varianza entre las letras O y L.

Sin embargo, cabe destacar una limitación teórica en esta selección: utilizar la varianza global no garantiza seleccionar los atributos con mayor poder discriminativo. Un atributo ideal para KNN es aquel que maximiza la varianza *inter-clase*, que presenta gran diferencia de intensidad entre O y L; mientras minimiza la varianza *intra-clase*, es decir, se mantiene constante entre las distintas tipografías de una misma letra. Si un píxel presenta alta varianza global únicamente debido a fluctuaciones internas en la clase O, su inclusión aportará ruido espacial en lugar de información útil para el cálculo de la distancia euclídea.

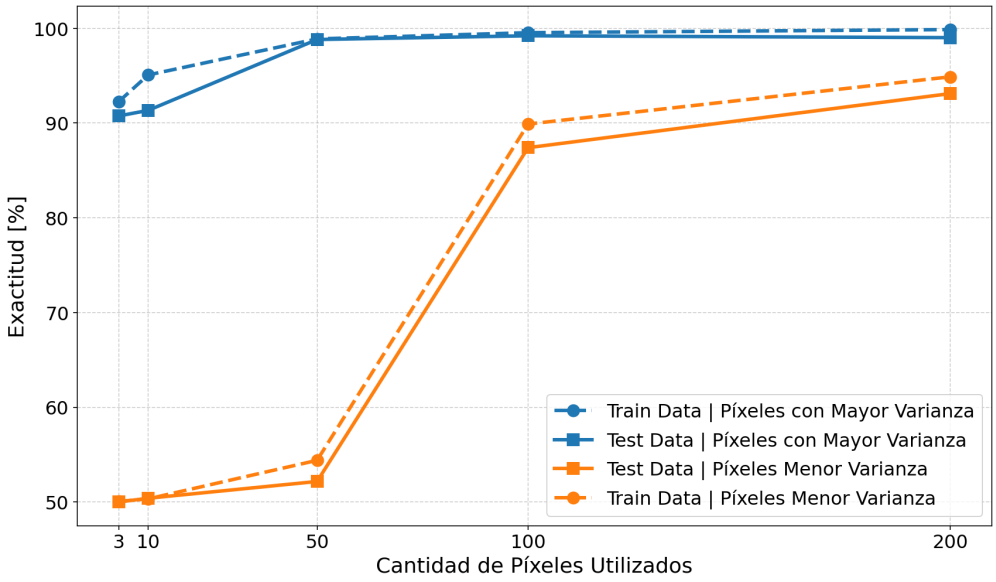

Figura 5: Exactitud del KNN con $K = 3$ para clasificar entre O y L en función de los píxeles utilizados. Líneas azules representan los píxeles de mayor varianza, las naranjas los de menor varianza. Además de la cantidad, también importan qué píxeles se eligen para entrenar al modelo.

3.2. Optimización del Hiperparámetro K

Una vez comprobada la eficacia de la selección de atributos, se decidieron utilizar 50 píxeles más variables para buscar la configuración óptima del modelo, por una cantidad de balance entre exactitud y complejidad del modelo. Se evaluó la exactitud variando la cantidad de vecinos K en el modelo desde 1 hasta 20.

La Figura 6 expone las curvas generada sobre el conjunto de datos de entrenamiento y de prueba, donde se enfrentan exactitud contra cantidad de vecinos. Cuando $K = 1$, el algoritmo memoriza el conjunto de entrenamiento, lo que es un claro síntoma de sobreajuste. A medida que K aumenta, la frontera de decisión se suaviza. El valor $K = 3$ representa el punto de equilibrio óptimo, ya que logra maximizar la exactitud sobre el conjunto de prueba sin un nivel de complejidad excesivo. Para valores de $K \geq 4$, el modelo comienza a presentar subajuste, degradando su capacidad predictiva. Notar que este gráfico se encuentra invertido horizontalmente respecto al de la Figura 1, pues tener menos vecinos corresponde a un modelo KNN más complejo[2].

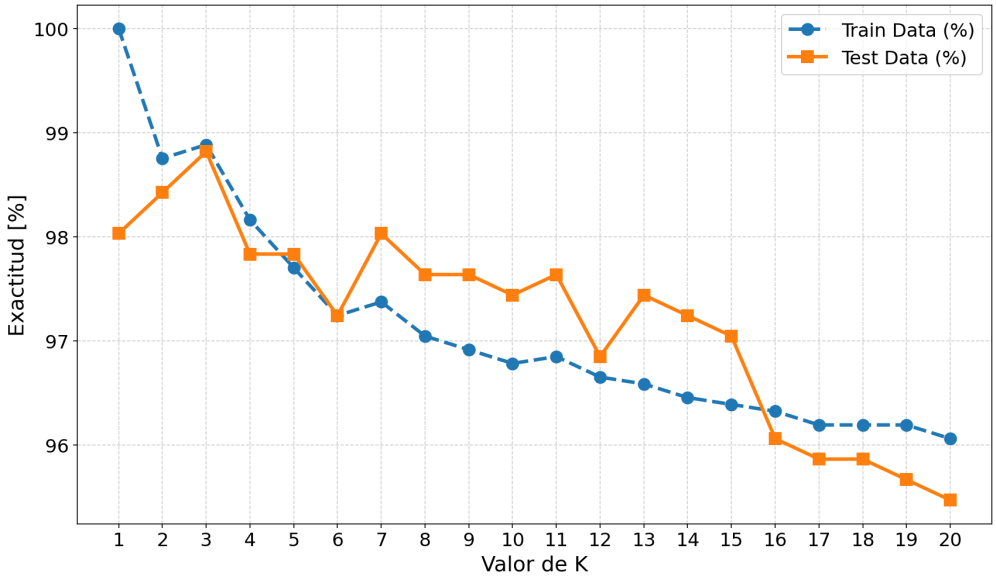

Figura 6: Valores para la exactitud al clasificar las letras O y L en función del valor de K para un modelo que usa los 50 píxeles con mayor varianza. $K = 3$ resulta ser el valor óptimo que maximiza resultados para el test data sin tener un modelo extremadamente simple ni complejo.

4. Clasificación Multiclase

Habiendo explorado la clasificación binaria, en esta sección se aborda el problema completo: clasificar las 26 letras del alfabeto simultáneamente. Dada la mayor complejidad de este nuevo espacio de decisión, se optó por implementar un modelo de Árboles de Decisión.

Inmediatamente surge la pregunta de qué tan complejo debe ser el modelo para clasificar de forma correcta las letras del alfabeto. Esto se responde mediante la Figura 7, donde después de haber separado el dataset en un conjunto de desarrollo y otro de Held-out en una proporción de 75/25, sobre el conjunto de desarrollo se evaluó la exactitud para un subconjunto de entrenamiento y otro de test, con proporción 80/20. Notar que más allá de la profundidad 10 se entra en zona de sobreajuste, por esta razón se decidió descartar al resto de profundidades.

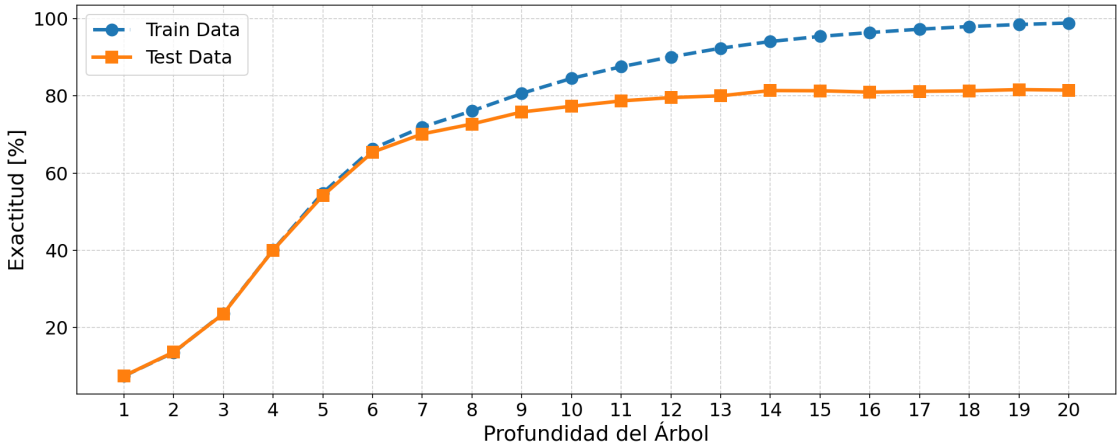

Figura 7: Exactitud del árbol de decisión al clasificar letras del abecedario según la profundidad. Rango ideal entre exactitud y complejidad sucede entre profundidad 8 y 10; a valores mayores se entra en sobreajuste (overfitting).

4.1. Selección de Hiperparámetros y Evaluación de la Exactitud

Conociendo el rango de máxima profundidad del árbol para no entrar una zona de sobreajuste, se quiso agregar un segundo hiperparámetro que ayude en el entrenamiento del modelo para así mejorar su capacidad de evaluar y clasificar caracteres del dataset. Para ello, se utilizó validación cruzada *k-folding* con $K = 5$ manteniendo la misma proporción de 80/20 usada en el análisis anterior. Se decidió aplicar los criterios de impureza como segundo hiperparámetro, por lo que se evaluó al Índice de Gini contra la Entropía en función de la maxima profundidad del árbol, permitiendo así conocer la exactitud media generada por cada criterio. Al aplicar *k-folding*, se utiliza la exactitud media para considerar cada una de las iteraciones de la evaluación cruzada.

Los resultados de este análisis se presentan en la Figura 8, allí se observa que la Entropía alcanza una mayor exactitud media que el criterio de Gini a partir de la profundidad 5. No obstante, determinar la configuración más adecuada exige un balance entre complejidad computacional y exactitud, por lo que la elección no se basa únicamente en el valor máximo absoluto, sino en el análisis de la tendencia de la curva. Si bien las profundidades 8, 9 y 10 presentan los mejores niveles de exactitud, se determinó que 9 es el valor óptimo. Esto se justifica al analizar la ganancia marginal: la mejora en la exactitud de la Entropía al pasar de la profundidad 8 a la 9 es de un destacable 2.9 %, mientras que el salto de la profundidad 9 a la 10 ofrece un incremento de apenas el 1.21 %. En consecuencia, se estableció que el punto de equilibrio ideal para el modelo se alcanza utilizando el criterio de Entropía con profundidad máxima 9 con exactitud del 78.91 %.

Utilizando la configuración óptima encontrada, se re-entrenó el árbol de decisión con la totalidad del conjunto de desarrollo y se lo evaluó, por única vez, sobre el conjunto Held-out. El modelo alcanzó una exactitud final del 78.51 %. La extrema similitud con la métrica de validación cruzada confirma que el modelo es robusto y generaliza correctamente sobre datos no vistos.

Para comprender la naturaleza del $\sim$20 % de clasificaciones erróneas, se analizó la Matriz de Confusión de la Figura 9. Se observó que el modelo es altamente preciso para caracteres con trazos únicos, como F y X, pero concentra sus fallas en pares que tienen una similitud elevada. El caso más crítico es la confusión entre I y J con 31 clasificaciones erróneas junto con T e I, seguido por las confusiones simétricas entre O y Q. Esto muestra que al modelo le cuesta diferenciar letras con alta similitud morfológica, concentrando los errores en esos pares de clasificaciones.

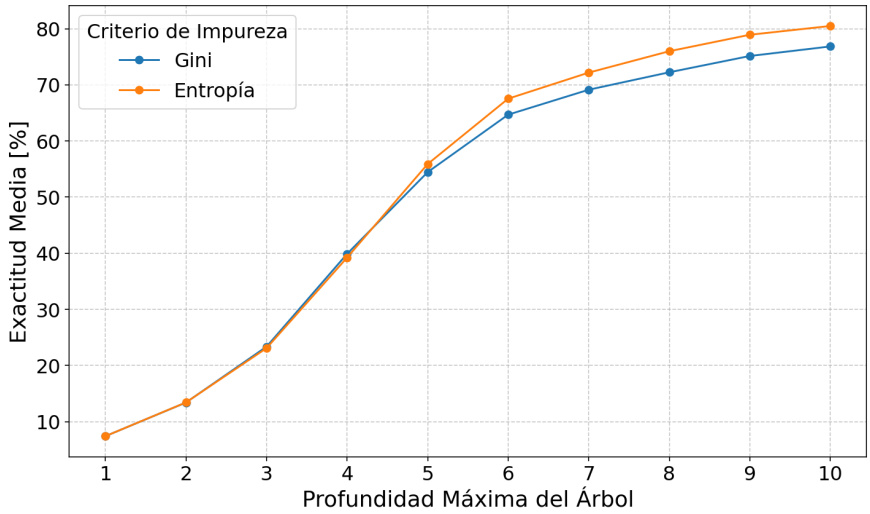

Figura 8: Comparación de criterios de impureza frente a la profundidad máxima del árbol de decisión. La curva azul representa la Ganancia de Gini, la curva naranja es la Ganancia de la Entropía.

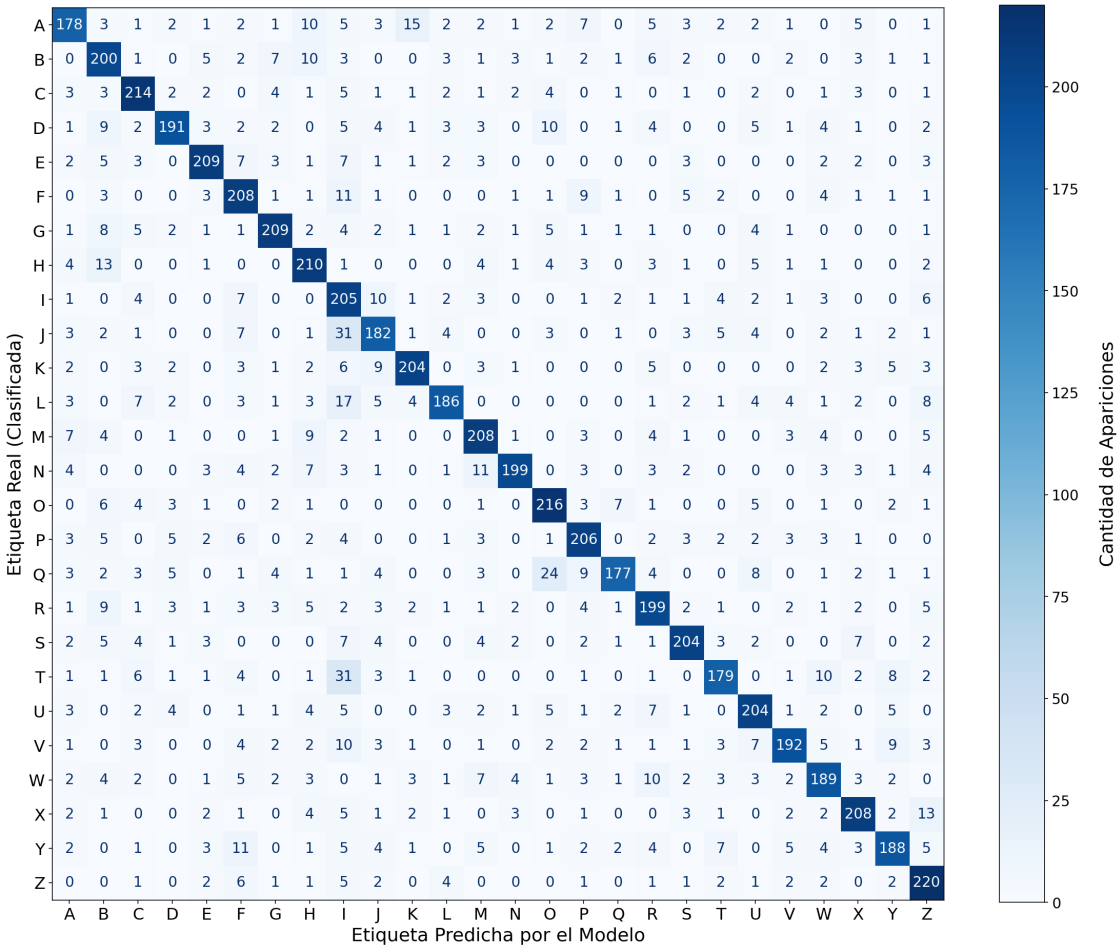

Figura 9: Matriz de confusión en el conjunto Held-out con profundidad 9 y entropía como criterio de impureza. Aciertos en la diagonal principal. Confusiones más repetidas son I con J, y T con I.

5. Conclusiones

El presente trabajo permitió abordar el problema de clasificación multiclase de imágenes tipográficas desde sus fundamentos exploratorios hasta la evaluación de modelos predictivos. A partir del Análisis Exploratorio de Datos (EDA), se demostró que si bien el balance perfecto de clases favorece el modelado, la similitud morfológica intrínseca entre ciertos pares de caracteres (tales como I vs. J u O vs. Q) impone un desafío natural para la separabilidad en un espacio de 784 dimensiones.

La metodología de partición de datos (Entrenamiento, Desarrollo y *Held-out*) garantizó una evaluación robusta y libre de sesgos metodológicos (*Data Leakage*). Durante la etapa de optimización, la búsqueda exhaustiva de hiperparámetros (*Grid Search*) mediante validación cruzada ($K = 5$) reveló que el criterio de partición basado en la **Entropía** (Ganancia de Información) superó al Índice de Gini para este dominio. Asimismo, limitar la profundidad máxima del árbol a 10 resultó ser el punto de equilibrio óptimo entre la capacidad predictiva y la prevención del sobreajuste (*overfitting*).

Finalmente, el modelo definitivo fue evaluado sobre el conjunto *Held-out*, logrando una **Exactitud del 80.03 %** en datos nunca antes vistos. La extrema cercanía entre esta métrica final y la obtenida durante la validación cruzada (80.48 %) certifica la excelente capacidad de generalización del modelo construido.

Si bien el rendimiento es altamente satisfactorio para un modelo de árbol simple, los errores observados en la matriz de confusión indican que para superar la barrera del 80 % de exactitud y distinguir los detalles morfológicos más sutiles, el trabajo futuro requeriría la implementación de métodos más complejos, tales como ensambles de árboles (*Random Forest*) o técnicas de extracción de características no lineales.

Referencias

[1] TM Mitchell. *Machine Learning*. McGraw-Hill International, 1997.

[2] Andreas C Müller and Sarah Guido. *Introduction to machine learning with Python: a guide for data scientists*. O'Reilly Media, Inc., 2016.